

PES UNIVERSITY

Department of Computer Science and Engineering

SOFTWARE ENGINEERING - LEVEL 3 MINI-PROJECT

Software Requirements Specification

Simple Shell - Command Line Interpreter

Item	Details
Project Title	Simple Shell - Command Line Interpreter
Group Number	Team 3
Course	UE24CS341A - Software Engineering
Semester	V Semester
Technology	C (POSIX / Linux)
Development Methodology	Agile
Team Members	Rekha Dhorigol (PES1UG24CS370) / Rohan Rampura Praveen Kumar (PES1UG24CS382) / Samika Ojha (PES1UG24CS412) / Sanjana Subrahmanya Aithal (PES1UG24CS421)
Version	1.0 - Requirements Baseline
Submission Format	PDF

Revision History

Version	Date	Description
1.0	2026-09-07	Initial requirements baseline

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Simple Shell (Command Line Interpreter) project undertaken by Team 3 as part of the Software Engineering Level 3 Mini-Project (UE24CS341A). It defines the functional, non-functional and security requirements for a modular command-line shell implemented in C for a Linux/Unix-like environment. This is Version 1.0 of the SRS, prepared at the Requirements stage of the project.

1.2 Intended Audience and Reading Suggestions

This document is intended for the project team (as developers and testers), the course faculty (as evaluators), and any reviewer assessing the completeness of the requirements. Section 2 gives an overall description of the product; Section 3 covers external interfaces; Section 4 presents the use case model; Section 5 details functional requirements by feature; Section 6 covers non-functional and security requirements; Section 7 covers other requirements; Section 8 gives the initial validation specification; Section 9 gives the project plan; Section 10 lists the tool stack; and the Appendices provide a glossary, internal data/field layouts and the requirement traceability matrix.

1.3 Product Scope

The Simple Shell is a lightweight, modular command-line interpreter written in C. It repeatedly prompts the user, reads and parses a command line, distinguishes built-in from external commands, and executes them using the POSIX process model. It supports foreground and background execution, input/output redirection, single and multiple command pipelines, command history and environment-variable management. The project's broader purpose is to give the team practical experience with core operating-system concepts (process creation, file descriptors, IPC, signals) while applying a disciplined software-engineering lifecycle (Agile planning, Git/GitHub, CI/CD, code review, testing and requirement traceability).

1.4 References

- SE Level 3 Mini-Project — Student Submission Guide & Complete Exemplar (Coffee Kiosk), PES University
- Jackfruit — Software Engineering Project Guidelines 2026, PES University, Subject Code UE24CS341A
- POSIX.1-2017 (IEEE Std 1003.1), relevant sections on process management, signals and I/O
- Team3_SimpleShell_Synopsis (Deliverable 1 - Synopsis)

2. Overall Description

2.1 Product Perspective

The Simple Shell is a new, self-contained, standalone application; it is not a follow-on to an existing product and has no dependency on a larger system. It replaces, for the scope of this course project, the day-to-day interactive use of a system shell such as bash for a defined subset of functionality. The system is composed of four cooperating modules: Command Interpretation (parsing and dispatch), Process Execution (process/job control), I/O Redirection & Pipelines, and Command History & Shell Environment. These modules interact through an in-process command/argument structure and the POSIX process/file APIs; there is no external server or network component.

2.2 Product Functions

- Read, tokenize and parse a command line into a command and its arguments.
- Classify a command as built-in or external and dispatch it accordingly.
- Execute a defined set of built-in commands (cd, pwd, echo, exit, export, unset, env, help, history).
- Create and manage child processes to execute external programs.
- Support foreground and background execution with correct process/signal handling.
- Redirect standard input/output using <, > and >>.
- Chain two or more commands using single or multiple pipelines (|).
- Store, display and recall previous commands (command history).
- Provide session-level access to environment variables.
- Detect and report invalid commands, malformed syntax and runtime errors without crashing.

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Students / Learners	Standard shell session; no elevated privileges	Use the shell interactively to understand OS/process concepts; not necessarily expert programmers.
Command-line Users	Standard shell session; no elevated privileges	Familiar with standard Unix/Linux shells; expect conventional shell behaviour.
Developers / Testers	Local build/run access; can inspect source and test output	Team members and reviewers extending or testing individual modules.
Faculty / Evaluators	Read/run access to the submitted repository and report	Assess functional correctness, SDLC evidence and individual contribution.

Note: the Simple Shell has a single runtime privilege level — it runs with the invoking OS user's own permissions and does not implement its own role-based access control (unlike a multi-user application). The distinctions above are about how each class interacts with the project, not in-shell authorization levels.

2.4 Scope

In Scope	Out of Scope
Interactive command prompt and input handling	Graphical user interface
Tokenization and parsing of commands/arguments	Remote shell access or network-based execution
Built-ins: cd, pwd, echo, exit, export, unset, env, help	Full-featured shell scripting language compatibility
Execution of external programs via the POSIX process model	Shell features not explicitly listed in approved requirements
Foreground and background command execution	Administrative/root-level system management
Input (<), output (>) and append (>>) redirection	Persistent (cross-session) command history
Single and multiple command pipelines ()	Multi-user session management
Current-session command history and environment-variable management	Shell scripting constructs (loops, conditionals, functions)

2.5 Operating Environment

- Target OS: Linux (POSIX-compatible); developed and tested on a standard Ubuntu/Linux distribution.
- Language/Runtime: C (C11), compiled with GCC.
- Execution mode: interactive terminal (TTY) session; no GUI.
- No database or external network service is required at runtime.

2.6 Design and Implementation Constraints

- Must use only POSIX system calls (fork, exec family, wait/waitpid, pipe, dup2, signal handling) — no shell-building libraries (e.g., readline is optional and must not replace core parsing logic).
- Development methodology is Agile only, tracked via GitHub/Jira with visible sprint history.
- Version control: Git/GitHub; CI/CD via Jenkins; static analysis via SonarQube or similar (per project guidelines).
- Memory safety: all dynamically allocated memory (tokens, argument vectors, history buffer) must be explicitly freed; no memory leaks in steady-state operation.
- The implementation must remain modular so each of the four functional areas (F1–F4) can be developed and unit-tested independently.

2.7 Assumptions and Dependencies

- Assumes execution on a single-user, single-host Linux environment with standard POSIX libraries available.
- Assumes external commands invoked by the user exist on the system PATH or are given as valid executable paths.
- Depends on the host OS's process, signal and file-descriptor APIs; no third-party runtime dependency is planned beyond the standard C library.
- If GNU Readline or a similar line-editing library is later adopted for history/editing convenience, it will be treated as an optional enhancement, not a core dependency.

3. External Interface Requirements

3.1 User Interfaces

- A textual command prompt (e.g., shell>) displayed on the terminal; the prompt is re-displayed after each command completes.
- Standard keyboard input for command entry; standard output/error streams for command results and error messages.
- Consistent error-message format: "shell: <command>: <reason>" for invalid/failed commands.
- Command history recall interface (e.g., a 'history' built-in listing prior commands with index numbers).

3.2 Software Interfaces

Interface	Description
POSIX process API	fork(), execvp()/execve(), waitpid(), used by the Process Execution module to create and manage child processes.
POSIX file/IPC API	open(), close(), dup2(), pipe(), used by the I/O Redirection & Pipelines module.
POSIX signal API	signal()/sigaction(), used for foreground/background job control and interrupt handling (e.g., SIGINT, SIGCHLD).
C standard library	String/memory functions (string.h, stdlib.h) used throughout for parsing and buffer management.
Environment API	getenv(), setenv(), unsetenv(), environ, used by the Command History & Shell Environment module.

3.3 Communications Interfaces

Not applicable. The Simple Shell is a local, single-host interactive application with no network communication requirements in this version.

4. Analysis Models

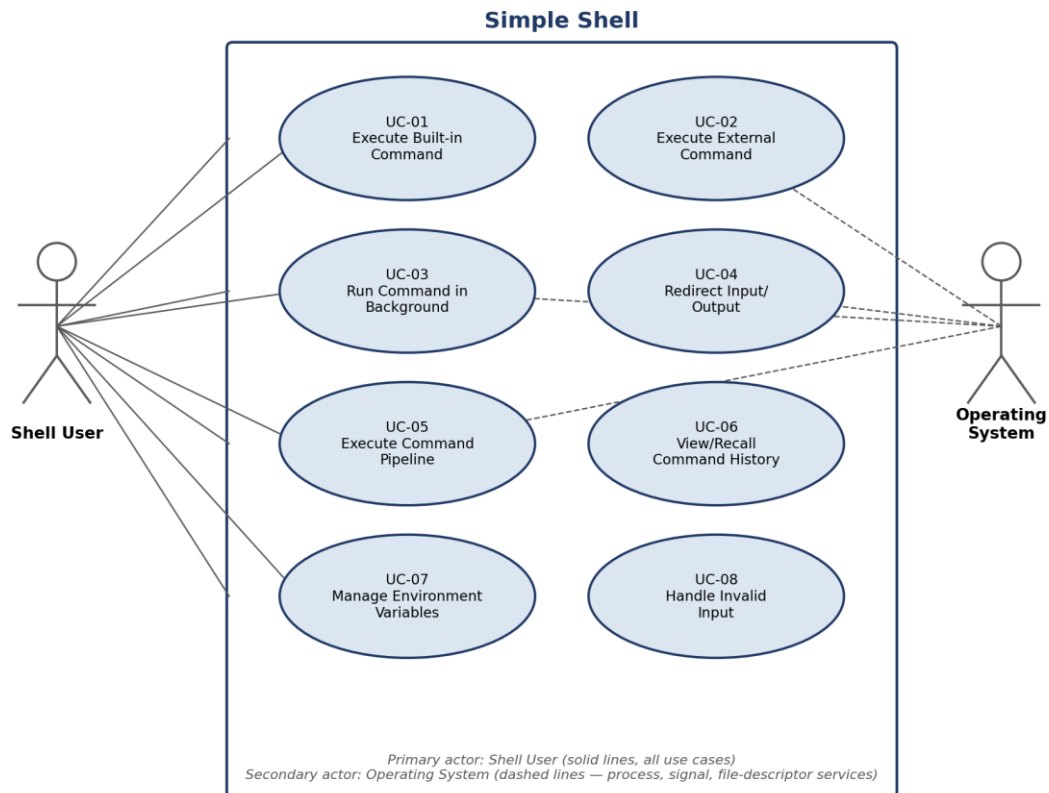
Actors: Shell User (primary actor interacting with the prompt) and the Operating System (external actor providing process, file and signal services).

4.1 Use Case Summary

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Execute Built-in Command	Shell User	Shell is running and prompt is displayed	Built-in command executed; result shown; prompt redisplayed
UC-02	Execute External Command	Shell User	Command exists on PATH or as valid path	Child process runs command; shell resumes after completion (foreground)
UC-03	Run Command in Background	Shell User	Command line ends with '&'	Child process starts; shell immediately redisplay prompt
UC-04	Redirect Input/Output	Shell User	Valid file path supplied with <, > or >>	Command's stdin/stdout bound to specified file
UC-05	Execute Command Pipeline	Shell User	Two or more commands separated by ' '	Output of each stage feeds into the next; final output shown
UC-06	View/Recall Command History	Shell User	At least one prior command has been entered	Prior commands listed or re-executed
UC-07	Manage Environment Variables	Shell User	Shell is running	Variable exported, unset, or listed via env
UC-08	Handle Invalid Input	Shell User	Malformed or unknown command entered	Descriptive error shown; shell continues without crashing

Note: A full use case diagram (actors, use cases and associations) and a sequence diagram for UC-02 and UC-05 will be produced as part of Deliverable 3 (Design), per the project's official flow of Requirements → Design → Implementation → Testing.

4.2 Use Case Diagram



Shell User is the primary actor and interacts with all eight use cases. The Operating System is a secondary actor supplying process, signal and file-descriptor services for the use cases that require them (UC-02, UC-03, UC-04, UC-05).

5. System Features

Requirements are organised by the four functional features defined in the Synopsis (F1–F4), one per team member, consistent with the approved functional ownership.

Feature	Owner	Focus
F1 — Command Interpretation & Core Built-in Commands	Rekha Dhorigol	Input handling, tokenization, parsing, classification, dispatch, core built-ins
F2 — Process Execution & Job Control	Rohan Rampura Praveen Kumar	External execution, foreground/background operation, process/signal handling
F3 — I/O Redirection & Pipelines	Samika Ojha	File-descriptor redirection and command pipelines
F4 — Command History & Shell Environment	Sanjana Subrahmanya Aithal	History and session-level environment operations

5.1 Command Interpretation & Core Built-in Commands (F1)

Owner: Rekha Dhorigol

5.1.1 Description and Priority

Reads a line of user input, tokenizes it into a command and arguments, classifies it as built-in or external, dispatches it to the correct subsystem, executes supported built-in commands directly, and reports invalid input. Priority: High — every other feature depends on this module's output.

5.1.2 Stimulus/Response Sequences

- User types a command and presses Enter → shell tokenizes the line, identifies the command, and either executes a built-in directly or forwards it to Process Execution (F2).
- User types an empty line → shell redisplay the prompt without action.
- User types an unrecognised command → shell reports an error and redisplay the prompt.

5.1.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-CI-01	The shell shall display a prompt and accept a line of input from standard input.	High	Functional test
FR-CI-02	The shell shall tokenize an input line into a command name and zero or more arguments, respecting quoted substrings.	High	Unit test
FR-CI-03	The shell shall classify each parsed command as a built-in or an external command before dispatch.	High	Unit test
FR-CI-04	The shell shall implement the core built-in commands cd, pwd, echo, exit and help.	High	Functional test
FR-CI-05	The shell shall display a descriptive error message and continue running when given an empty, malformed, or unrecognised command.	High	Negative test
FR-CI-06	The shell shall not terminate or crash on invalid input, oversized input, or unexpected argument counts.	High	Robustness/fuzz test
FR-CI-07	The exit built-in shall terminate the shell process cleanly, releasing all allocated resources.	Medium	Functional + memory test

5.2 Process Execution & Job Control (F2)

Owner: Rohan Rampura Praveen Kumar

5.2.1 Description and Priority

Executes external programs by creating child processes and replacing their program image, supports foreground and background execution, and handles relevant process signals. Priority: High.

5.2.2 Stimulus/Response Sequences

- User enters an external command → shell forks a child, the child execs the target program, the parent waits (foreground) or continues immediately (background).
- User presses Ctrl+C during a foreground command → signal is delivered to the foreground process group, not the shell itself.
- A background job finishes → shell reports completion (e.g., on next prompt) without blocking the user.

5.2.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-PE-01	The shell shall create a child process using fork() for each external command to be executed.	High	Unit test
FR-PE-02	The shell shall replace the child process's program image using an exec-family call with the parsed command and arguments.	High	Functional test
FR-PE-03	The shell shall wait for a foreground command to complete before redisplaying the prompt.	High	Functional test
FR-PE-04	The shell shall support background execution when a command line ends with '&', returning control to the prompt immediately.	High	Functional test
FR-PE-05	The shell shall report the exit status of a foreground command when it terminates abnormally (e.g., by signal).	Medium	Functional test
FR-PE-06	The shell shall reap terminated background child processes to prevent zombie processes.	High	Integration test
FR-PE-07	The shell shall forward SIGINT (Ctrl+C) to the foreground process group without terminating the shell itself.	High	Signal-handling test

5.3 I/O Redirection & Pipelines (F3)

Owner: Samika Ojha

5.3.1 Description and Priority

Redirects a command's standard input/output to files and connects two or more commands through pipelines using file descriptors and inter-process communication. Priority: High.

5.3.2 Stimulus/Response Sequences

- User supplies '> file' or '>> file' → command's stdout is redirected to the file (truncate or append respectively).
- User supplies '< file' → command's stdin is bound to the file's contents.
- User separates two or more commands with '|' → the shell creates a pipe per junction and connects each stage's stdout to the next stage's stdin.

5.3.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-IO-01	The shell shall redirect a command's standard output to a specified file when '>' is present, truncating the file if it exists.	High	Functional test
FR-IO-02	The shell shall append a command's standard output to a specified file when '>>' is present.	High	Functional test
FR-IO-03	The shell shall redirect a command's standard input from a specified file when '<' is present.	High	Functional test
FR-IO-04	The shell shall support a pipeline of two commands connected by a single ' ', passing the first command's output as the second's input.	High	Integration test
FR-IO-05	The shell shall support pipelines of three or more commands chained by multiple ' ' operators.	Medium	Integration test
FR-IO-06	The shell shall report an error and skip execution when a redirection target file cannot be opened (e.g., permission denied).	High	Negative test
FR-IO-07	The shell shall close all unused pipe file descriptors in each process to avoid descriptor leaks and hangs.	High	Integration/resource test

5.4 Command History & Shell Environment (F4)

Owner: Sanjana Subrahmanya Aithal

5.4.1 Description and Priority

Stores and displays previously entered commands and provides session-level access to environment variables through supported shell operations. Priority: Medium-High.

5.4.2 Stimulus/Response Sequences

- User enters a command → shell appends it to an in-memory history list (bounded size).
- User invokes the history built-in → shell lists prior commands with index numbers.
- User invokes export/unset/env → shell updates or lists the current session's environment variables.

5.4.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-HE-01	The shell shall record each successfully parsed command line in an in-memory history buffer of a fixed maximum size.	Medium	Unit test
FR-HE-02	The shell shall display the recorded history, in order, with a numeric index, via a history built-in.	Medium	Functional test
FR-HE-03	The shell shall discard the oldest entry when the history buffer is full and a new command is recorded.	Medium	Unit test
FR-HE-04	The shell shall allow a user to set or update an environment variable for the current session via export.	High	Functional test
FR-HE-05	The shell shall allow a user to remove an environment variable for the current session via unset.	Medium	Functional test
FR-HE-06	The shell shall list all currently set environment variables via the env built-in.	Medium	Functional test
FR-HE-07	Environment variable changes made in the shell session shall be visible to child processes it subsequently creates.	High	Integration test

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Category	Measurable Requirement	Verification
NFR-01	Responsiveness	The shell shall display the prompt and accept new input within 100 ms of a foreground command completing.	Performance test
NFR-02	Startup time	The shell shall be ready to accept input within 200 ms of launch on the reference test environment.	Performance test
NFR-03	Pipeline throughput	A pipeline of up to 5 stages shall introduce no more than 50 ms of additional latency versus direct piping in bash for equivalent commands.	Benchmark test

6.2 Safety Requirements

- The shell shall not perform destructive actions (e.g., deleting files) except through the external commands the user explicitly invokes; it introduces no hidden destructive built-ins.
- Background and orphaned child processes shall be properly reaped so they do not persist as zombies after the shell exits.

6.3 Security Requirements

ID	Security Requirement	Validation Approach
SEC-01	The shell shall not execute a command via a shell-interpolation function (e.g., <code>system()</code>) that could enable shell-injection through unsanitised input; commands shall be passed as discrete argument vectors to <code>exec</code> .	Code review + negative test
SEC-02	The shell shall validate and bound all user-supplied input lengths before copying into fixed-size buffers, preventing buffer overflows.	Static analysis + fuzz test
SEC-03	The shell shall not expose or log the values of sensitive environment variables (e.g., credentials) beyond what the user explicitly requests via <code>env</code> .	Code review
SEC-04	The shell shall drop or ignore signals that could be used to crash or hijack the parent shell process from a malicious child (e.g., unexpected <code>SIGCHLD</code> handling edge cases).	Security/negative test
SEC-05	File redirection targets shall be opened with explicit, minimal permissions and shall not silently overwrite files outside the user's intent (e.g., <code>'>'</code> vs <code>'>>'</code> distinction strictly enforced).	Functional + negative test
SEC-06	All dynamic memory used for parsing and history shall be bounds-checked to prevent use-after-free and double-free defects.	Static analysis (e.g., Valgrind/ASan)

6.4 Software Quality Attributes

Attribute	Requirement
Maintainability	Business/parsing logic shall be separated into identifiable modules (parser, executor, redirection/pipe, history/env) with unit tests per module.
Testability	Each module shall expose functions that can be unit-tested independently of live terminal input (e.g., by injecting a command string).
Reliability	The shell shall not crash or leave the terminal in a corrupted state after a malformed command, failed exec, or interrupted pipeline.
Portability	The shell shall build and run on any standard POSIX-compliant Linux distribution using GCC, without distribution-specific dependencies.
Usability	Error messages shall be specific enough (command name + reason) that a first-time user can identify the mistake without external documentation.

6.5 Business Rules / Domain Rules

ID	Rule
SR-01	A command line is only dispatched to Process Execution (F2) if Command Interpretation (F1) has successfully classified it as a valid external command.
SR-02	Redirection and pipeline parsing (F3) is resolved before process creation, so file descriptors are correctly bound prior to exec.
SR-03	Only the shell process itself may modify its own history buffer and exported-variable table; child processes cannot alter the parent shell's session state.
SR-04	A background job ('&') shall never block the shell from redisplaying its prompt, regardless of the job's own runtime.

ID	Rule
SR-05	cd changes only the current shell process's working directory; it has no effect on the parent terminal outside the shell.
SR-06	Command history is current-session only unless persistence across sessions is explicitly approved as a later enhancement.

7. Other Requirements

- Build requirement: the project shall provide a Makefile that builds the shell with a single 'make' command on a standard Linux environment.
- Reuse: standard C library and POSIX system calls only; no reuse of an existing shell's source code.
- Documentation requirement: a README shall document setup, build and run instructions, consistent with the project guidelines.
- Legal/licensing: the project shall be released under an open license (e.g., MIT) suitable for academic submission; no third-party proprietary code shall be included.

8. Validation Specification — Initial Planning

At this Requirements stage, validation planning is intentionally at the planning level only — it states how each requirement category will eventually be validated, not the final test results. The final Validation Specification (updated after Design, and again after Implementation) will reference concrete test cases and results.

Requirement Category	Planned Validation	Example
Functional (F1–F4)	Black-box functional tests per feature, executed against defined inputs/outputs	Valid built-in execution; correct pipeline chaining
Security (SEC-01–SEC-06)	Negative/security testing, static analysis (e.g., cppcheck), memory-safety tooling (Valgrind/ASan)	Injection attempt via crafted argument; buffer-boundary input
Performance (NFR-01–NFR-03)	Timed benchmark tests comparing shell response/latency against defined thresholds	Prompt redisplay latency after foreground command
Reliability/Robustness	Fuzz and negative testing with malformed, empty and oversized input	Random/garbage input, extremely long command lines
Unit-level (per module)	Automated unit tests with coverage reporting per module (parser, executor, redirection, history/env)	Tokenizer unit tests, environment table unit tests
Resource safety	Runtime checks for file-descriptor leaks and zombie processes	Repeated pipeline execution under load; background job storms

9. Project Plan

This is the initial, requirements-stage project plan (Agile, iteration-based). It will be refined once Design (Deliverable 3) is complete.

9.1 Work Breakdown Structure

Work Package	Tasks	Owner	Estimated Effort (person-days)	Output
WP1	Requirements gathering, use cases, SRS baseline	All	4	SRS v1.0
WP2	Architecture + module decomposition + UML	All (led by Rekha)	4	Architecture/Design doc
WP3	Command Interpretation & core built-ins (F1)	Rekha	6	Working feature + unit tests
WP4	Process Execution & job control (F2)	Rohan	6	Working feature + unit tests
WP5	I/O Redirection & pipelines (F3)	Samika	6	Working feature + unit tests
WP6	Command history & environment (F4)	Sanjana	6	Working feature + unit tests
WP7	Integration, CI/CD, static analysis, security validation	All	6	Test/validation evidence
WP8	Final system validation, defect fixing, report, demo	All	4	Final report + working demo

9.2 Schedule (Sprint Mapping)

Aligned with the six-sprint plan in the Synopsis: Sprint 1 covers WP1–WP2; Sprints 2–4 cover WP3–WP6 (one functional area brought to a working, tested state per sprint, in parallel across owners); Sprint 5 covers WP7; Sprint 6 covers WP8.

10. Tool Stack

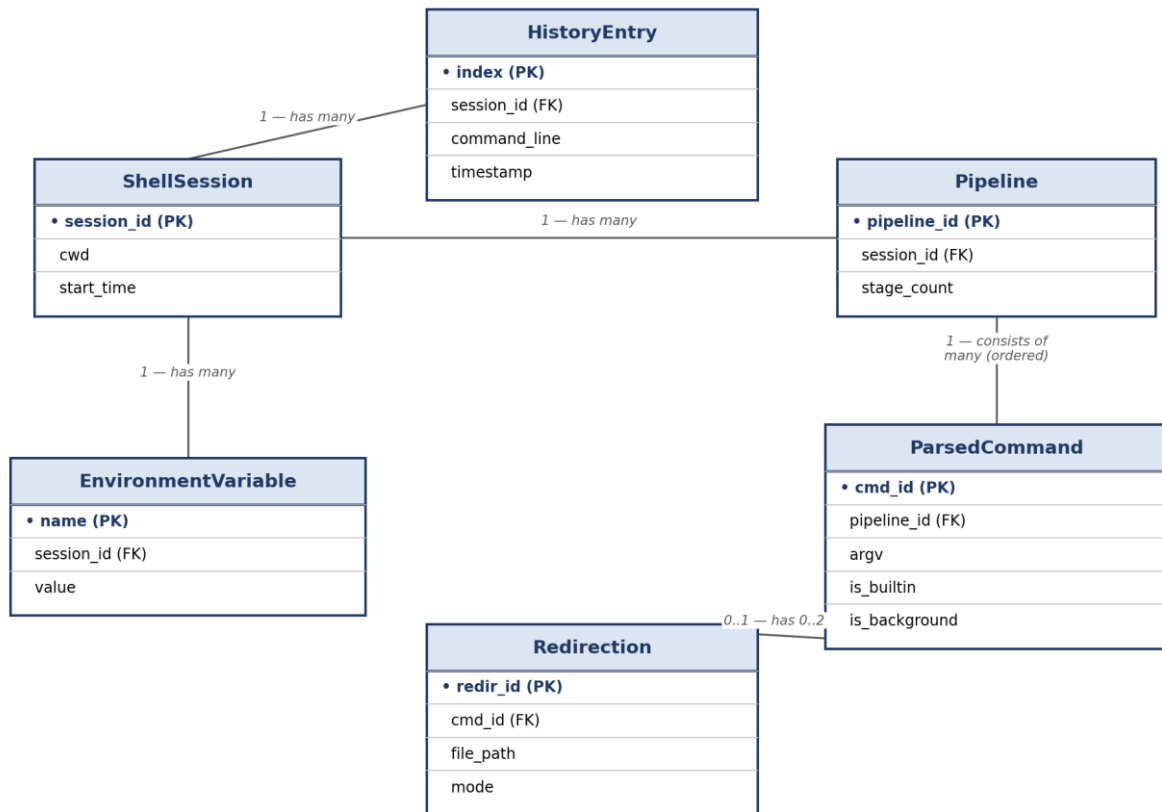
Activity	Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL, commit history
Agile planning	Jira or GitHub Projects	Backlog, user stories, sprint board
Build	GCC + Make	Makefile, build log
CI/CD	Jenkins	Pipeline screenshot/log
Containerization	Docker	Dockerfile + build/run evidence
Testing	GoogleTest / Check / custom C test harness	Test results
Static analysis	SonarQube or a suitable C static-analysis tool (e.g., cppcheck)	Analysis report
Memory/robustness	Valgrind or AddressSanitizer	Leak/UB report
Design	Draw.io / StarUML / PlantUML	UML diagrams

Appendix A: Glossary

Term	Definition
Built-in command	A command implemented and executed directly within the shell process (e.g., cd, exit), without creating a child process.
External command	A command executed by creating a child process and replacing its image with an external executable found on PATH or given as a path.
Foreground execution	Execution mode in which the shell waits for the invoked command to complete before accepting new input.
Background execution	Execution mode (command line ending in '&') in which the shell does not wait for the invoked command to complete.
Redirection	Binding a command's standard input or output to a file instead of the terminal, using <, > or >>.
Pipeline	A sequence of two or more commands connected so that the standard output of one becomes the standard input of the next, using ' '.
Job control	The set of shell mechanisms for managing foreground/background processes and delivering signals to them.
Zombie process	A terminated child process whose exit status has not yet been collected by its parent via wait()/waitpid().
Environment variable	A named value in the process environment (accessible via getenv/setenv/envIRON) inherited by child processes.

Appendix B: Field Layouts

The Simple Shell has no external database; the following ER-style diagram and field tables document its key in-memory data structures instead.



In-memory data model — the Simple Shell has no persistent database; these entities and relationships exist only within a single running shell session (see Appendix B for field layouts).

B.1 History Entry

Field	Type	Description	Mandatory
index	int	Sequential position of the command in the history buffer.	Y
command line	char[]	The raw command line as entered by the user.	Y
timestamp	time_t	Time the command was entered (optional, for display).	N

B.2 Parsed Command Token

Field	Type	Description	Mandatory
argv	char*[]	NULL-terminated argument vector for the command.	Y
is_builtin	bool	Whether the command matches a recognised built-in.	Y
input_file	char*	Path for input redirection ('<'), if present.	N
output_file	char*	Path for output redirection ('>' or '>>'), if present.	N
append_mode	bool	True if '>>' was used instead of '>'.	N
is_background	bool	True if the command line ended with '&'.	Y

B.3 Environment Variable Entry

Field	Type	Description	Mandatory
name	char*	Variable name (e.g., PATH).	Y
value	char*	Variable value.	Y

Appendix C: Requirement Traceability Matrix

At this Requirements stage, Architecture/Design/Code/Test references are placeholders (TBD) to be completed as Deliverables 3 and 4 progress; the Requirement ID and Description columns are finalised now.

Req ID	Brief Description	Arch. Ref	Design Ref	Code Ref	Test Case ID	Sys. Test ID
FR-CI-01	Display prompt, accept input line	TBD	TBD	TBD	TBD	TBD
FR-CI-02	Tokenize input line	TBD	TBD	TBD	TBD	TBD
FR-CI-03	Classify built-in vs external	TBD	TBD	TBD	TBD	TBD
FR-CI-04	Implement core built-ins	TBD	TBD	TBD	TBD	TBD
FR-CI-05	Error message on invalid input	TBD	TBD	TBD	TBD	TBD
FR-CI-06	No crash on invalid input	TBD	TBD	TBD	TBD	TBD
FR-CI-07	Clean exit	TBD	TBD	TBD	TBD	TBD
FR-PE-01	fork() per external command	TBD	TBD	TBD	TBD	TBD
FR-PE-02	exec-family program replacement	TBD	TBD	TBD	TBD	TBD
FR-PE-03	Wait for foreground completion	TBD	TBD	TBD	TBD	TBD
FR-PE-04	Background execution ('&')	TBD	TBD	TBD	TBD	TBD
FR-PE-05	Report abnormal exit status	TBD	TBD	TBD	TBD	TBD
FR-PE-06	Reap terminated background children	TBD	TBD	TBD	TBD	TBD
FR-PE-07	Forward SIGINT to foreground group	TBD	TBD	TBD	TBD	TBD
FR-IO-01	Output redirection ('>')	TBD	TBD	TBD	TBD	TBD
FR-IO-02	Append redirection ('>>')	TBD	TBD	TBD	TBD	TBD
FR-IO-03	Input redirection ('<')	TBD	TBD	TBD	TBD	TBD
FR-IO-04	Two-stage pipeline	TBD	TBD	TBD	TBD	TBD
FR-IO-05	Multi-stage pipeline	TBD	TBD	TBD	TBD	TBD
FR-IO-06	Error on inaccessible redirection target	TBD	TBD	TBD	TBD	TBD
FR-IO-07	Close unused pipe descriptors	TBD	TBD	TBD	TBD	TBD
FR-HE-01	Record command in history buffer	TBD	TBD	TBD	TBD	TBD
FR-HE-02	Display history via built-in	TBD	TBD	TBD	TBD	TBD
FR-HE-03	Discard oldest entry when full	TBD	TBD	TBD	TBD	TBD
FR-HE-04	export sets/updates variable	TBD	TBD	TBD	TBD	TBD
FR-HE-05	unset removes variable	TBD	TBD	TBD	TBD	TBD
FR-HE-06	env lists variables	TBD	TBD	TBD	TBD	TBD
FR-HE-07	Env changes visible to children	TBD	TBD	TBD	TBD	TBD
SEC-01	No shell-injection via exec	TBD	TBD	TBD	TBD	TBD
SEC-02	Bounded input, no buffer overflow	TBD	TBD	TBD	TBD	TBD
SEC-03	No unintended exposure of sensitive env vars	TBD	TBD	TBD	TBD	TBD
SEC-04	Safe signal handling from children	TBD	TBD	TBD	TBD	TBD
SEC-05	Explicit, minimal redirection permissions	TBD	TBD	TBD	TBD	TBD
SEC-06	No use-after-free/double-free	TBD	TBD	TBD	TBD	TBD
NFR-01	Prompt responsiveness $\leq 100\text{ms}$	TBD	TBD	TBD	TBD	TBD
NFR-02	Startup time $\leq 200\text{ms}$	TBD	TBD	TBD	TBD	TBD
NFR-03	Pipeline throughput overhead $\leq 50\text{ms}$	TBD	TBD	TBD	TBD	TBD